EEG-BCI Finger/Action Project

User Manual (End-to-End)

Prepared for onboarding new researchers

January 1, 2026

Contents

1 Scope and goals

This manual explains the complete end-to-end workflow implemented in the EEG-BCI repository:

- recording or importing EEG sessions,
- validating event labels,
- extracting fixed-length windows for machine learning,
- training a compact multi-head deep model (action head + finger head),
- evaluating accuracy and calibration,
- generating reports (including Deepchecks outputs),
- running a live demo UI + backend using LSL streams,
- improving live stability via postprocessing (smoothing, hysteresis, threshold gating, adjacency assist),
- troubleshooting the most common runtime issues on macOS (OpenMP duplicate runtime).

Important: This manual is written to match the repo structure and scripts as used in your terminal logs:

```
1b_extract_windows.py
2_train_model.py
3_evaluate_model.py
4_generate_reports.py
demo_backend/
demo_app/
```

2 System overview

2.1 High-level dataflow

1. **Raw EEG + event logs** are recorded per session (subject id, experiment hash, timestamps).
2. **Processed features** are computed and aligned with events.
3. **Event validation** checks schema integrity and distribution sanity.
4. **Window extraction** generates:
   - `eeg_windows.csv` (tabular window metadata/labels),
   - `eeg_windows.npz` (model-ready tensors and metadata).
5. **Training (Step 2)** fits a CNN+LSTM multi-head model and saves:
   - `finger_action_model.pt` (PyTorch weights),
   - `scaler.save` (channel normalizer),
   - `test_predictions.npz` (probabilities + labels for reproducible evaluation).

6. **Evaluation (Step 3)** loads the model, normalizer, and either recomputes predictions or reuses `test_predictions.npz` to compute:
   - action accuracy,
   - finger accuracy on non-REST samples,
   - calibration metrics (ECE),
   - plots and confusion matrices.
7. **Reports (Step 4)** build subject and cross-subject summaries.
8. **Live demo** streams LSL EEG into the backend, runs inference, applies postprocessing, and visualizes decisions in a Vite/React UI.

2.2 Labeling problem: two heads

The model predicts two related targets per window:

- **Action head:** `action_id` includes REST and non-REST actions (e.g., open vs close).
- **Finger head:** `finger_id` includes NONE (0) plus specific fingers. In evaluation, finger accuracy is typically reported **only for non-REST action windows**.

3 Repository layout (what to read first)

Recommended onboarding reading order:

1. **`docs/`** (any design notes, data schema notes).
2. **`utils/label_schema.py`**: canonical IDs and display names for actions and fingers.
3. **`utils/sequence_data.py`**: how `eeg_windows.npz` is created, loaded, normalized, and split.
4. **`models/cnn_lstm_finger_action_net.py`**: network architecture and tensor shapes.
5. **`2_train_model.py`**: training loop, loss definitions, and saved artifacts.
6. **`3_evaluate_model.py`**: evaluation metrics, calibration, confusion matrices, plots.
7. **`demo_backend/`**: inference engine, live LSL source, replay source, and WebSocket tick schema.
8. **`demo_app/`**: UI controls, connection logic, and diagnostics display.

4 Quickstart for new researchers

This is the shortest path to reproducing a complete run on an existing dataset.

4.1 Environment

Activate the conda environment you already use:

```
conda activate base
```

On macOS, evaluation may crash due to duplicate OpenMP runtime loading (common when mixing PyTorch/NumPy/Scipy MKL/OpenMP builds). Use the known workaround:

```
export KMP_DUPLICATE_LIB_OK=TRUE
export OMP_NUM_THREADS=1
export MKL_NUM_THREADS=1
```

4.2 Validate events

```
python 5_validate_events.py
```

Typical output includes counts by action and finger, plus a PASS/WARN decision.

4.3 Extract windows

```
python 1b_extract_windows.py
```

Expected artifacts:

```
eeg_windows.csv
eeg_windows.npz
```

4.4 Train baseline model

Example "strong" run that produced improved metrics in your logs:

```
python 2_train_model.py --epochs 60 --batch-size 32 --rest-weight 0.2
```

4.5 Evaluate

```
export KMP_DUPLICATE_LIB_OK=TRUE
export OMP_NUM_THREADS=1
export MKL_NUM_THREADS=1
python 3_evaluate_model.py
```

4.6 Generate reports

```
python 4_generate_reports.py
```

5 Step 1: Data and events

5.1 Session metadata

Sessions commonly write a JSON metadata file (e.g., `session_meta.json`) to preserve:

- `subject_id`
- `experiment_hash`
- recording time
- mode flags (if used)

This metadata is used by downstream scripts to name outputs and log experiments.

5.2 Events file

During **STEP 1** recording, the script `1_stream_and_record.py` writes two key "per session" CSV artifacts into `data/processed/`:

```
data/processed/<subject>_<timestamp>_events.csv
data/processed/<subject>_<timestamp>_eeg_features.csv
```

What `*_events.csv` contains. This is the time-aligned event log used to label the EEG. Each row corresponds to a discrete user action (e.g., a keypress or GUI event) with at least a timestamp and the categorical labels needed to train: `action_id` (REST/OPEN/CLOSE) and `finger_id` (NONE/THUMB/INDEX/MIDDLE/RING/PINKY). The exact column set is enforced by the validator (see below).

What `*_eeg_features.csv` contains. Despite the name "features", this file is the cleaned *per-sample* EEG time series written after ICA-based artifact attenuation (see header comments in `1_stream_and_record.py`). The four numeric columns are:

- `ch1` = TP9
- `ch2` = AF7
- `ch3` = AF8
- `ch4` = TP10

Each row is one timestamped sample; downstream preprocessing groups these samples into 64-sample windows to form the model input tensor described in Section "What is a window?".

5.3 Event validation (5_validate_events.py)

Validation checks usually include:

- required columns present,
- optional columns (e.g. `session_mode`) may be missing and produce warnings,

- event counts and class distributions are sensible,
- the REST class exists but is not dominating the dataset,
- action and finger IDs are within the label schema.

Why validation matters:

- Window extraction uses event spans to label windows. Broken timestamps create mislabeled windows.
- Class imbalance (too many REST or too few per finger) can inflate accuracy or collapse learning.

6 Step 1b: Window extraction (1b_extract_windows.py)

6.1 What is a window?

A *window* is the atomic input unit for the neural model and for live inference. In this repo it is defined exactly as:

$$X \in \mathbb{R}^{(N,\,64,\,4)}$$

where:

- N = number of windows (samples) in the dataset or batch.
- 64 = timesteps per window. At 256 Hz sampling, this is $64/256 = 0.25$ s of EEG.
- 4 = EEG channels, ordered as: `TP9`, `AF7`, `AF8`, `TP10`.

Why this matters. This ordering must remain consistent from (1) streaming/recording, to (2) feature extraction/windowing, to (3) training and evaluation, and finally to (4) live LSL inference. If you permute channels at any stage, the model will see the wrong spatial patterns and performance will collapse.

How windows are created. A continuous time series is segmented into overlapping or non-overlapping chunks of exactly 64 samples per channel (implementation lives in the data preprocessing utilities). Each window is paired with:

- an **action label** y_{action} in {REST, OPEN, CLOSE}, and
- a **finger label** y_{finger} in {NONE, THUMB, INDEX, MIDDLE, RING, PINKY}.

Sanity check. When debugging, print one window and verify:

- `window.shape == (64,4)`
- channel 0 corresponds to TP9, channel 1 to AF7, channel 2 to AF8, channel 3 to TP10.

6.2 Window selection rules

Your extraction summary shows windows are kept/dropped based on:

- **no-overlap:** windows that do not overlap any labeled event span,
- **artifact-overlap:** (0 in your run) windows overlapping artifact segments,
- **guard-band:** windows too close to boundaries (helps prevent leakage around transitions),
- **invalid label / short segment:** sanity constraints.

6.3 Outputs

`eeg_windows.csv` is used for quick inspection and debugging distributions. `eeg_windows.npz` is the authoritative tensor input to training and evaluation.

7 Step 2: Training (2_train_model.py)

7.1 Model architecture: CNN + LSTM multi-head

A typical CNN+LSTM design for EEG windows is:

- **CNN frontend:** learns local temporal filters and channel mixing.
- **LSTM backend:** integrates across the window to capture dynamics.
- **Two heads:**
  - action head outputs logits over actions,
  - finger head outputs logits over fingers.

7.2 Normalization

Training fits a *channel normalizer* on training data only and applies it to train/test:

```
scaler.save
```

This prevents data leakage and ensures stable feature scaling.

7.3 Losses and masking

Two cross-entropy losses are used:

$$\mathcal{L} = \mathcal{L}_{action} + \lambda \mathcal{L}_{finger}$$

Finger loss is masked to non-REST examples:

$$\mathcal{L}_{finger} = \mathrm{CE}(f(x), y_{finger}) \text{ only where } y_{action} \neq \texttt{ACTION\_REST}.$$

This matches the semantic meaning: when the action is REST, finger identity is not meaningful.

7.4 REST reweighting

Your best run used:

```
--rest-weight 0.2
```

This typically means REST examples contribute less to the action loss. Why it helps:

- REST can be "easy" and abundant. The model may over-optimize REST confidence and underfit non-REST discrimination.
- Lowering REST weight forces the model to allocate capacity to non-REST decisions, often improving finger accuracy and non-REST action accuracy.

7.5 Saved artifacts

After training completes:

- `finger_action_model.pt`: model weights
- `scaler.save`: fitted normalizer
- `test_predictions.npz`: cached test probabilities and labels (if produced by your training script)
- `logs/experiments/<hash>_train_config.json`: hyperparameters and input shape

8 Step 3: Evaluation and calibration (3_evaluate_model.py)

8.1 Accuracy metrics

Action accuracy:

$$\text{Acc}_{action} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}[\hat{y}^{(i)}_{action} = y^{(i)}_{action}]$$

Finger accuracy is computed only on non-REST subset:

$$\text{Acc}_{finger} = \frac{1}{N'} \sum_{i: y^{(i)}_{action} \neq REST} \mathbb{1}[\hat{y}^{(i)}_{finger} = y^{(i)}_{finger}]$$

8.2 Confidence and softmax

Logits z are converted to probabilities by softmax:

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Confidence is the maximum probability:

$$c = \max_k p_k$$

8.3 Expected calibration error (ECE)

ECE measures the gap between confidence and accuracy binned by confidence. For bins B_m:

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \left|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\right|.$$

Lower is better. Your improved run achieved (example):

- Action ECE ≈ 0.0346
- Finger ECE (non-REST) ≈ 0.0456

8.4 Using test_predictions.npz for reproducibility

If `test_predictions.npz` exists, evaluation can skip model inference and reuse saved:

- `action_probs`, `finger_probs`
- `y_action`, `y_finger`
- `test_indices` (for traceability back to the full dataset order)

This guarantees that metrics match prior runs even if code changes, as long as label schema is unchanged.

8.5 Optional postprocessed metrics

The repo includes a stateful postprocessor for live stability. Offline evaluation can optionally apply it sequentially (ordered by time) to compute "smoothed" accuracies. This is useful because the postprocessor is causal and depends on previous frames.

9 Step 4: Report generation (4_generate_reports.py)

Report generation typically:

- collects metrics and plots per subject,
- writes subject-specific folders under `reports/subjects/<subject_id>/`,
- optionally generates cross-subject comparisons,
- detects calibration files and warns if none are present (as in your log).

10 Deepchecks report (offline data quality)

You provided and zipped:

```
reports/deepchecks/deepchecks_eeg_report.html
```

Deepchecks-style reports usually surface:

- label leakage risk,

- train/test distribution shift,
- class imbalance,
- feature drift or outliers,
- correlation and redundancy among channels/features,
- segments with unusually high variance (artifacts),
- confusion "hot spots" indicating systematic confusions between specific fingers.

When interpreting the report, prioritize findings that directly match observed confusions in your finger confusion matrix.

11 Live demo system

11.1 Components

- **demo_backend**: Python server that handles model loading, LSL acquisition, replay, inference, and WebSocket streaming to the UI.
- **demo_app**: Vite/React frontend that connects to the backend, displays predictions, and exposes controls.
- **LSL (Lab Streaming Layer)**: network discovery and streaming of EEG samples (e.g., `PetalStream_eeg`).

11.2 Starting the backend

From repo root:

```
conda activate base
python -m demo_backend.server
```

11.3 Starting the UI

From `demo_app`:

```
cd demo_app
npm run dev
```

If port 5173 is taken, Vite will automatically choose another (e.g., 5174):

```
Local: http://localhost:5174/
```

11.4 LSL stream selection

To list available streams:

```
python - <<'PY'
from pylsl import resolve_streams
streams = resolve_streams()
print("Found", len(streams), "streams\n")
for i,s in enumerate(streams):
    print(f"[{i}] name={s.name()} type={s.type()} "
          f"source_id={s.source_id()} uid={s.uid()} "
          f"ch={s.channel_count()} srate={s.nominal_srate()}")
PY
```

Your example showed:

```
name=PetalStream_eeg type=EEG ch=5 srate=256.0
```

A robust selection rule is to pick the first stream whose name contains `"eeg"` (case-insensitive). If none is found, the backend should emit:

```
no eeg stream found
```

and keep live mode idle so the UI disconnects gracefully.

12 Live postprocessing (stability features)

In live use, raw model predictions can jitter due to noise, boundary transitions, or low confidence. The repo adds a **stateful postprocessor** that runs after the model and before actuation/UI display.

12.1 Features

Feature	What it does
Smoothing (vote)	Maintains a sliding window of recent class IDs and outputs the majority class. Also averages probabilities in that window for confidence reporting.
Smoothing (EMA)	Exponentially averages probabilities. With window size N, the alpha used is: $\alpha = \frac{2}{N+1}$.
Threshold gating	If action confidence $< T_{action}$, force REST (no actuation). If finger confidence $< T_{finger}$, force finger NONE (0).
Hysteresis	Prevents rapid flips between action states (e.g., open/close). Requires a new action to persist for K frames before committing. Also supports a margin above threshold to switch faster.
Adjacency assist	If the top two finger probabilities are close (gap below Δ) and the second choice is adjacent to the last finger in the anatomical chain, prefer the adjacent finger to reduce flicker.
Diagnostics	Exposes `decision_reason`, raw and smoothed IDs, committed IDs, and `frames_in_state` to the UI for debugging.

12.2 How to tune postprocessing in practice

Start with conservative defaults:

- smoothing: enabled, vote, window 5
- action threshold: 0.75
- finger threshold: 0.75
- hysteresis: enabled in live mode, frames 3
- adjacency assist: enabled

Then tune in this order:

1. Increase thresholds if you see many false positives (random non-REST detections at rest).
2. Increase smoothing window if predictions are noisy, but note this adds latency.
3. Increase hysteresis frames if action toggles rapidly.
4. Disable adjacency assist if it causes systematic bias toward a neighbor finger.

13 UI controls: what to click (operator checklist)

This section describes the UI workflow for running tests and checks. Labels may vary slightly, but the control intent is stable.

13.1 Open the UI

1. Start backend: `python -m demo_backend.server`
2. Start UI: `cd demo_app && npm run dev`
3. In the terminal, read the printed URL (e.g., http://localhost:5174/) and open it in your browser.

13.2 Connection and mode selection

In the UI:

1. Locate the **Mode** selector. Choose:
   - **Replay** to test on a saved dataset (`eeg_windows.npz`).
   - **Live** to connect to LSL and run real-time inference.
   - **Idle** to stop streaming.
2. If in **Replay** mode, set the **Replay path** to `eeg_windows.npz` unless testing another file.
3. Click **Start** (or equivalent). Confirm status shows "Connected" and ticks begin updating.
4. For **Live** mode, confirm an EEG LSL stream exists. If not, the UI should show a status warning such as "no eeg stream found" and disconnect.

13.3 Model stability controls

Still in the UI, in the control panel:

1. Enable **Smoothing**.
2. Choose **Method: Vote** initially.
3. Set **Window N** to 5 (increase if too jittery).
4. Enable **Hysteresis** in live mode; in replay mode it may be off by default (unless the user explicitly enabled it).
5. Set **Hysteresis frames** to 3.
6. Set **Action threshold** to 75%.
7. Set **Finger threshold** to 75%.
8. Enable **Adjacency assist**.

13.4 Decision diagnostics

Use the **Decision** panel:

- Confirm `decision_reason` is usually `vote_commit` or `ema_commit`.
- If you see `below_threshold` frequently during intended movement, thresholds are too high *or* the model is underconfident.
- If you see `hysteresis_hold` frequently during intentional transitions, reduce hysteresis frames or margin.

14 Troubleshooting

14.1 OpenMP duplicate runtime crash on macOS

Symptom:

```
OMP: Error #15: Initializing libomp.dylib, but found libomp.dylib already initialized.
zsh: abort  python 3_evaluate_model.py
```

Workaround (known, but "unsafe" per OpenMP message):

```
export KMP_DUPLICATE_LIB_OK=TRUE
export OMP_NUM_THREADS=1
export MKL_NUM_THREADS=1
python 3_evaluate_model.py
```

Longer-term fix options for maintainers:

- Ensure a single OpenMP runtime is used (avoid linking two copies via Conda + Homebrew).
- Prefer consistent builds of numpy/scipy/pytorch (all conda-forge or all defaults) to reduce runtime duplication.

14.2 Vite port already in use

If you see:

```
Port 5173 is in use, trying another one...
Local: http://localhost:5174/
```

Just open the shown URL.

14.3 LSL connects to wrong stream

If LSL attaches to gyro/ppg/etc. instead of EEG:

- Print resolved streams (script shown earlier).
- Ensure backend stream selection filters by name containing "eeg".
- If multiple EEG streams exist, extend selection to prefer `type=EEG` and/or channel count.

15 Improving accuracy: practical playbook

Your improved run achieved:

- Action accuracy: 67.52%
- Finger accuracy (non-REST): 62.16%

To push further, prefer changes in this order (highest ROI first):

15.1 Data improvements

- **Increase samples per finger and per action**, especially for weaker fingers (often ring/pinky).
- **Balance the class distribution**: keep non-REST examples comparable across fingers.
- **Reduce label noise**: enforce consistent event timing and guard-bands.
- **Artifact handling**: mark and exclude high-motion/no-contact segments.

15.2 Training improvements

- **REST weighting**: continue tuning `-rest-weight`. Too low can increase false positives at rest.
- **Learning rate schedule**: consider step decay or cosine annealing to improve final convergence.
- **Regularization**: mild dropout or weight decay can reduce overconfidence and improve generalization.
- **Early stopping**: stop when validation metrics plateau (prevents overfitting).

15.3 Evaluation improvements

- Always report both **raw** and **postprocessed** metrics for live readiness.
- Use `test_predictions.npz` to ensure reproducibility when comparing model changes.

15.4 Postprocessing improvements

- If accuracy is fine but live feels unstable, tune thresholds and hysteresis before retraining.
- If the model is underconfident, consider temperature scaling as a calibration step.

16 Appendix A: Common terminal recipes

16.1 End-to-end run

```
conda activate base
python 5_validate_events.py
python 1b_extract_windows.py
python 2_train_model.py --epochs 60 --batch-size 32 --rest-weight 0.2

export KMP_DUPLICATE_LIB_OK=TRUE
export OMP_NUM_THREADS=1
export MKL_NUM_THREADS=1
python 3_evaluate_model.py

python 4_generate_reports.py
```

16.2 Zip reports for sharing

```
zip -r deepchecks_report.zip reports/deepchecks
```

16.3 Run the demo

```
# Terminal 1: backend
conda activate base
python -m demo_backend.server

# Terminal 2: UI
cd demo_app
npm run dev
```

17 Appendix B: Glossary

Term	Meaning
REST	No intended movement; used as a background class.
Window	Fixed-length EEG segment used as one training example.
Head	Output branch of a neural network for a specific target (action vs finger).
ECE	Expected calibration error, measures confidence vs accuracy mismatch.
LSL	Lab Streaming Layer, a protocol for streaming biosignals.
Hysteresis	A stability mechanism requiring persistence before switching state.

EMA	Exponential moving average of probabilities for smoothing.